LAlgo: what we need to do

Team guide | Switch and case revision

1. What matters now

The lexer and EBNF checker support `SWITCH`, `CASE`, `DEFAULT`, and `BREAK`. Run `make syntax` to check the switch sample. This guide condenses the four recordings and the checking rubrics. Short, timestamped excerpts support the requirements; full transcripts stay in the meeting notes.

READY means the stated portion exists in the current source or documentation. The lexer passes **33/33 local tests**, and the syntax checker passes **13/13 CLI tests**.

FOCUS means missing work, partial coverage, or a decision we still need to settle. Use the syntax checker to demonstrate the implemented grammar.

Language decisions for checking

1. **Use `WHILE` for single and nested loops.** The EBNF covers both forms.
2. **Keep language features through iteration only.** Review future-reserved `ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT`; do not expand them into extra supported constructs for this scope.
3. **Show `SWITCH` during checking.** Use a reference selector, at least one case, one final default, and `BREAK;` at the end of every arm.

Ma'am: "Kung FOR or WHILE, bahala kayo. Pero ang rule niya, dalawa."
Recording 04, 05:24–05:46. She then names single and nested loops.

Dates stated in the meeting

Date, 2026	Milestone
12 November	Submit syntax proposal: grammar, examples, and derivations. Face-to-face by default.
19 November	Asynchronous project work.
26 November	Lexical checking 2 and syntax checking 1.
3 December	Asynchronous project work.
10 December	Syntax checking 2, semantics checking 1, final project checking and final exam.

Team priority: scope decisions → grammar and per-rule derivations → parser and diagnostics. The iteration limit concerns language features; syntax and semantic checking milestones still apply. Submission time/platform and detailed scoring weights are not supplied in these materials. The semantic guide is now included on page 6.

2. Lexical analyzer: what already works

READY Recognition and output are implemented. The rows below map to the 11 lexical-rubric areas. They describe lexical behavior, not syntax or execution.

#	Area	Existing evidence — READY
1	Input restriction	Accepts `.lal` / `.LAL`; rejects other extensions before opening the source. Rejects NUL-containing input.
2	Identifiers	Letter first; then letters, digits, or underscores. Case-sensitive.
3	Keywords	The unified DFA recognizes `SWITCH`, `CASE`, `DEFAULT`, and `BREAK` as control-flow keywords.
4	Reserved words	Reserved spellings and separate future-reserved token classes exist. Their final scope needs review below.
5	Constant values	Five categories: integer, decimal, character, string, Boolean. `TRUE`/`FALSE` count as one category.
6	Noise words	Recognizes `SET` and `THE`.
7	Comments	Recognizes `//` line comments and `/* ... */` block comments; retains them in the listing.
8a	Arithmetic: 7	`+ - * / % DIV ^`. Assignment `=` is counted separately.
8b	Boolean: 9	Logical: `AND OR NOT`. Relational: `== != < > <= >=`.
9	Delimiters/brackets	Recognizes `( ) [ ] { }`, comma, semicolon, colon, and dot. Pairing/nesting checks belong to parsing.
10	Invalid tokens	Emits `INVALID` and lexical line/column diagnostics for documented malformed lexemes.
11	Output text file	Writes `Symbol Table.txt`: token occurrences, classifications, positions, and lexemes; includes comments/invalid entries.

Ma'am: "Sa lexical analyzer n'yo, na ang output ay symbol table."
Recording 04, 00:50–00:57. Retain the lexical output file.

Focus before presenting this as complete

- **Show the keyword tokens and the colon.** Use `switch_sample.lal` and its symbol table during checking.
- **Resolve rubric interpretation.** The extension filter works, but global uniqueness of `.lal` is unverified. Confirm the intended keyword/reserved distinction and nine-Boolean-operator classification.
- **Reconcile previous feedback.** Local tests do not establish that every item from Ma'am's previous checking has been addressed.

Demo evidence to keep: a valid `.lal` sample, a rejected `.txt`, all token categories, malformed input, and the generated `Symbol Table.txt`. The blank rubric scores remain Ma'am's assessment.

3. Syntax forms for checking

READY The EBNF and syntax checker cover the listed forms. The four switch rules are R42–R45 in Chapter 3.

Family	Coverage Ma'am expects	Local status
Input /1	One input construct; its rule design is up to us.	**READY** Input rule
Output /3	String only; variable value only; string plus variable value.	**READY** Three forms
Assignment /3	Variable to variable; constant to variable; expression result to variable.	**READY** Three forms
Selection	The three IF forms and `SWITCH` with labeled cases and a default.	**READY** IF and SWITCH
Iteration /2	Single loop and nested loops, using one chosen loop kind.	**READY** WHILE, both forms
Declaration	Required for us because it is already proposed. Discuss single-variable, list, and initialized forms.	**READY** Three forms

Ma'am: "One example for every rule in that statement."
Recording 02, 01:02–01:08. Each rule needs its own matching example.

What each applicable rule needs for 12 November

1. **Formal grammar components:** terminals, nonterminals, productions, and a grammar start symbol. The lexer's `q0` is not that start symbol.
2. **A distinct matching example:** three examples of a single IF do not cover three different conditional forms. Cover additional applicable rules too.
3. **Leftmost and rightmost derivations:** apply the actual written productions until the example is obtained. Show both complete derivations and one shared parse tree.

READY Documented groundwork only: token vocabulary, expression examples, operator meanings, and intended precedence/associativity already exist in the specification. Reuse these decisions, then encode them in the grammar. They are not implemented evaluation.

Keep these distinctions clear. `a = b + 3` is plain assignment; `INTEGER a = b + 3` is declaration with initialization. The declaration count is not fixed on the screen; Ma'am tentatively discusses three forms. Her printf examples do not force C-style formatting in LAlgo.

4. Demonstrate the syntax checker

READY Available foundation: the lexer stores token type, lexeme, line, and column in memory and creates EOF. Ma'am permits an **in-memory handoff**, one grammar terminal at a time. Continue generating `Symbol Table.txt` for the lexical rubric.

What to show during checking

1. Scan the switch sample.

Run `./build/lalgo syntax_proposal/switch_sample.lal`. Show the four keyword tokens, colon delimiters, and the generated symbol table.

2. Check the syntax.

Run `make syntax`. The checker reads the C lexer terminal stream and checks the same EBNF printed in the proposal. It accepts nested switches and requires a final default.

3. Show the derivations and a diagnostic.

Use R42–R45 for the derivations and parse trees. Remove a case colon or arm terminator in a scratch copy to show a syntax error with its line, column, and expected token.

4. Implement panic recovery.

The checker currently reports the farthest mismatch. The proposed recovery stage will resume checking later statements after an error.

Ma'am: "The error message must be brief and concise, but it will further give the programmer the reason why there is such an error."
Recording 04, 07:38–07:48. Include a useful reason, not just "syntax error."

Optional artifacts — not automatic blockers

Syntax transition diagram and parsing table are optional in the checking guide. Their usefulness depends on the approach we choose. Existing lexical DFA diagrams and the `DELTA` transition table are different artifacts.

Team handoff

1. **Bring the current proposal.** Chapter 3 contains the grammar and the 45 examples, each with two derivations and one shared parse tree.
2. **Show the checking evidence.** Run the lexer and syntax checker on the switch sample. Use a malformed scratch copy for diagnostics.
3. **Prepare the next stage.** Define selector and case types, check duplicate case labels, and add recovery that resumes after syntax errors.

Where this comes from. The 5 November recordings 01–04, the lexical, syntax, and semantic checking guides, and Adriel's reported iteration scope limit. Current status reflects the switch and case revision. Quoted excerpts are lightly edited machine-assisted transcriptions; recording numbers follow the supplied files 1–4, with timestamps restarting per file. The iteration scope limit was reported by Adriel without a verified audio timestamp. Full transcripts and audit notes remain in the unified meeting notes. Course scores are reported separately from local validation.

5. Ma'am's syntax checking table

These are the criteria shown on screen. The **Existing** and **Point** columns are left blank, just as in the guide. Our progress assessment is on pages 2–4.

Criteria	Existing	Point
1. Input must be read one by one from the symbol table (one token at a time)		
2. A parser algorithm was implemented Algorithm: Recursive Descent Parsing		
3. Transition Diagram		
4. Parsing table		
5. Input Statement /1		
6. Output Statement /3		
7. Assignment Statement /3		
8. Condition Statement /3		
9. Iterative Statement /2		
10. Declaration Statement		
11. Error Recovery Method used:		
12. Error Messages		
Total:		
Percentage (total score/total items)		

Read this with recording 04. Recursive descent is an example; items 3–4 are optional. Declaration applies when it is in the proposal, as it is for LAlgo. Iteration /2 means single and nested loops of one chosen kind. In-memory terminal input is allowed.

Source: Ma'am's Syntax Analyzer screenshot, also explained in recording 04. The written table is reproduced separately from our implementation status; no points or percentage have been assigned here.

6. Ma'am's semantic checking table

FOCUS Semantic analysis still needs to be built. We have token recognition and source positions, but no semantic-rule checker or semantic symbol-table updates yet.

Criteria	Existing	Point
1. Implementation of Semantic Rules: Evidence: must have a semantic rule for every syntactic rule		
Semantic Errors: (Basic)		
2. Input Statement Data Type Mismatch Variable Not Declared		
3. Output Statement Data Type Mismatch Variable Not Declared		
4. Assignment Statement Data Type Mismatch Variable Not Declared		
5. Condition Statement Data Type Mismatch Variable Not Declared		
6. Iterative Statement Data Type Mismatch Variable Not Declared		
7. Declaration Statement Data Type Mismatch Variable Not Declared Double Declaration Warning: Variable declared but not used		
8. Additional Semantic Error(s) (Optional)		
9. Error Recovery		
10. Error Messages		
11. Changes in Symbol Table value		
Total Grade		

What we need to add for the semantic milestone

- **A semantic rule for every syntax rule.** Define legal types, declarations, identifier use, and how each construct changes semantic state.
- **Checks and useful diagnostics.** Cover the listed type/declaration errors, duplicate declarations, unused-variable warnings, and recovery. Extra error categories are optional.
- **A semantic symbol table with updates.** The current token-occurrence listing is groundwork, not this deliverable. Define and demonstrate the value/state changes required by the guide.

Source: the Semantic Analyzer screenshot supplied by Adriel after the recordings. No accompanying semantic explanation/audio was supplied. The table is transcribed as shown; exact type rules, update behavior, and scoring must be defined or clarified. Existing/Point cells remain blank.